

סדנה מתקדמת בתכנות

פרק 06 : מחרוזות

חזרה : ייצוג המערך בזיכרון

- המערך מיוצג בזיכרון כרצף של איבריו, כלומר אם הגדרנו מערך בגודל size וכל איבר מטיפוס int אזי המערך ייוצג בזיכרון כרצף של size int-ים.
- שם המערך (a) מתייחס לאזור המערך כולו, המתחיל באיבר הראשון במערך. כאשר נדרשת כתובת - שם המערך מייצג את כתובת האיבר הראשון במערך.
- בכדי לגשת לאיבר במערך משתמשים באופרטור [].
למשל $a[2]$ - הינו האיבר בעל אינדקס 2 במערך a, כלומר האיבר השלישי.

מערך תווים ומחרוזת

- אין בשפת C טיפוס string (מחרוזת) אבל יש מושג "מחרוזת"

- מערך תווים הוא באופן כללי מערך רגיל. אבל כתנאי פרטי אפשר לייצג מערך תווים כמחרוזת ולהתייחס אליה כמשתנה אחד מורכב.

```
char ch_arr[]={ 'a','r','r','a','y' }; // not string
```

```
char str[]={ 'a','r','r','a','y','\0' }; // string
```

- הגדרה שנייה מסתיימת בתו '0' אשר מציין סוף של מחרוזת ונותן לקומפיילר סימן להגבלה.
הגדרה שנייה שקולה ל-

```
char str[]="array"; // string
```

מערך תווים ומחרוזת

- מחרוזת בשפת C היא מערך תווים (char) שבסופו התו '\0'.
התו '\0' (תו אפסי – Null Character) הוא תו מיוחד שהערך ה-ASCII שלו הוא 0.
- כמו כל מערך המחרוזת מעוברת כפרמטר לפונקציה ע"י כתובת התו הראשון שלה. אין צורך להעביר פרמטר נוסף הקובע את גודל המערך כי '\0' מציין סוף המחרוזת וקובע אורך שלה.
- **מחרוזת קבועה (constant string):** סדרת תווים בין " " כפולות, למשל "Hello". טקסט בתוך printf היינו דוגמה של מחרוזת קבועה.
- המחרוזת "a" שונה מהתו 'a'.
"a" הוא מערך תווים בעל גודל 2 לפחות בו איבר מס' 0 הוא 'a' ואיבר מס' 1 הוא '\0'.

```
char ch_arr[]={'a','r','r','a','y'};
char str[]="array"; // char str[]={'a','r','r','a','y','\0'};
```

```
printf(" print ch_arr: %s \n", ch_arr);
printf(" print str: %s \n", str);
```

```
print ch_arr: array
print str: array
Press any key to continue . . .
```

```
char ch_arr[]={ 'a','r','r','a','y' };
char str[]="array"; // char str[]={ 'a','r','r','a','y','\0' };
int i, lng=sizeof(ch_arr);
for(i=0; i<lng; i++)
    printf("ch[%d]=%c \n", i, ch_arr[i]);
printf("str: ");
for(i=0; str[i]!='\0'; i++)
    printf("%c ", str[i]);
printf("\n");
```

```
ch[0]=a
ch[1]=r
ch[2]=r
ch[3]=a
ch[4]=y
str: a r r a y
Press any key to continue . . .
```

- הדפסת מחרוזות באמצעות הפונקציה `printf()` מתבצעת ע"י שימוש בסימן הבקרה **%s** והעברת כתובת המחרוזת. חלק המערך שמודפס הוא מהתו שכתובתו הועברה ועד ה- `'\0'`.

```
#include<stdio.h>
void main()
{
    char p[]="hello";
    printf("%s \n", p);
}
```

הפלט:

hello

```
char str1[]="array",  
      str2[]="string";
```

```
printf("%s\n%s\n",str1,str2);
```

שקול ל-

```
puts(str1);  
puts(str2);
```

```
array  
string  
Press any key to continue . . .
```

קלט מחרוזות

```
char str1[30], str2[30];  
printf("Enter first line: "); scanf("%s",str1);  
printf("Enter second line: "); scanf("%s",str2);  
printf("\nFirst line: %s\nSecond line: %s\n",str1,str2);
```

לא צריך & כי שם מחרוזת הוא כבר כתובת!!

גודל מקסימאלי של קלט לכל
מחרוזת הוא 29 (צריך להשאיר
מקום ל- '\0')
משתמש אחראי לכך שלא
לעבור גבול פיסי של המערך!

Enter first line: John
Enter second line: Rabi

First line: John
Second line: Rabi
Press any key to continue . . .

```
char str1[10], str2[5];  
printf("Enter first line: "); scanf("%9s",str1);  
rewind(stdin); // fflush() in some other versions  
printf("Enter second line: "); scanf("%4s",str2);  
printf("\nFirst line: %s\nSecond line: %s\n",str1,str2);
```

הגנה: עכשיו משתמש לא יכול
לעבור גבול פיסי של המערך
(צריך גם להשאיר מקום ל- '\0')

Enter first line: 123456789012345
Enter second line: abcde

First line: 123456789
Second line: abcd
Press any key to continue . . .

```
char str1[30], str2[30];  
printf("Enter first line: "); scanf("%s",str1);  
printf("Enter second line: "); scanf("%s",str2);  
printf("\nFirst line: %s\nSecond line: %s\n",str1,str2);
```

**scanf פעילה עבור מחרוזות ללא רווחים!!
אחרת קולטת שתי מילים לשתי מחרוזות.**

```
Enter first line: John Rabi  
Enter second line:  
First line: John  
Second line: Rabi  
Press any key to continue . . .
```

```
char str1[30], str2[30];  
printf("Enter first line: "); gets(str1);  
printf("Enter second line: "); gets(str2);  
printf("\nFirst line: %s\nSecond line: %s\n",str1,str2);
```

```
Enter first line: John Rabi  
Enter second line: Mike Smith  
First line: John Rabi  
Second line: Mike Smith  
Press any key to continue . . .
```

```
char str1[30], str2[30];  
printf("Enter first line: "); gets(str1);  
printf("Enter second line: "); gets(str2);  
printf("\nFirst line: %s\nSecond line: %s\n",str1,str2);
```

אבל הקוד נותן אזהרה!
הסיבה לכך היא ש- `gets` אינה מבצעת בדיקת גבולות.
היא לא בודקת אם קלט המשתמש נמצא בתוך מספר
הבתים שצוין לאחסון במערך התווים.
קלט גדול מגודל המערך עלול להוביל לגלישה!
לכן פונקציה זו הוצאה משימוש – כלומר היא מיושנת
ומומלץ לא להשתמש בה.

חלופות עבור `gets()`

```
char buffer[BUFFERSIZE];  
gets(buffer);  
fgets(buffer, sizeof(buffer), stdin);
```

מגבלה



■ לעומת `gets`, הפונקציה לא מחליפה `'\n'` ע"י `'\0'` אלא מוסיפה `'\0'` אחרי `'\n'`

חלופות עבור `gets()`

```
char buffer[BUFFERSIZE];  
gets(buffer);  
gets_s(buffer, sizeof(buffer));
```

מגבלה



- הפונקציה אינה מוגדרת בכל הגרסאות של שפת C
- אם הקלט חורג מהמגבלה שנקבעה – שגיאת ריצה

שימוש בספריית string.h

תוך ספרייה string.h נמצא מספר פונקציות לעיבוד מחרוזות.

פונקציות בסיסיות:

`strlen(char *str)` - מחזירה אורך המחרוזת

`strcpy(char *str1, char *str2)` - מעתיקה מחרוזת שנייה לתוך ראשונה

`strcmp(char*str1, char *str2)` - משווה שתי מחרוזות

...

שימוש בספריית string.h

```
char str[50];  
strcpy(str, "Hello!");  
printf("length of str: %d \n", strlen(str));  
printf("str: %s \n", str);
```

```
length of str: 6  
str: Hello!
```

```
str[4]='\0';  
printf("length of str: %d \n", strlen(str));  
printf("str: %s \n", str);
```

```
length of str: 4  
str: Hell
```

שימוש בספריית string.h

```
printf(" Jim, John -> %d \n", strcmp("Jim", "John"));  
printf(" John, Jim -> %d \n", strcmp("John", "Jim"));  
printf(" John, John -> %d \n", strcmp("John", "John"));
```

```
Jim, John -> -1  
John, Jim -> 1  
John, John -> 0
```

השוואה לקסיקוגרפית

הגדרת מחרוזת

■ מחרוזת היא מערך לכל דבר:

`char str[] = "hello";`
`char str[6] = "hello";` שקול ל-



■ אפשר להתייחס לכל תו של מחרוזתה בנפרד

■ אפשר לעדכן תוכן של מחרוזת: כל תו בנפרד או בפונקציות על מחרוזות (`strcpy`, `scanf` וכו')

■ אפשר לאתחל מחרוזת רק בהצהרה

■ אסור לשנות מצביע שבאמצעותו מוגדר מערך (`str`) כי הוא קבוע.
ניסיון לשנות `str` – שגיאת קומפילציה!

הגדרת מחרוזת

כמו כן אפשר להתייחס אל מחרוזת כאל מצביע, לדוגמה:

```
char *str = "hello", c;
```

**צריך להתייחס למחרוזת קבועה
במשפט כמו לכתובת התו הראשון שלה**



עבור הגדרה זו של `str` המחשב מקצה שטח זיכרון ל-`str` ולמחרוזת "hello" ומציב ל-`str` את כתובת התו הראשון של המחרוזת.

אפשר לשנות `str` במהלך ריצה כי הוא מוגדר כמשתנה. למשל:

```
str++; ; str=&c ; str="good-bye"
```

אפשר להתייחס לכל תו של מחרוזתה בנפרד כמו לאיבר מערך

אסור לשנות תוכן מחרוזת כי היא מחרוזת קבועה שנמצאת במקום מסוים של זיכרון בו אי-אפשר לעשות עדכונים.

ניסיון לשנות תוכן המחרוזת – שגיאת ריצה!

- לא ניתן להציב מחרוזת לאחר ההגדרה, לדוגמה:

```
char str[6];  
str = "hello"; /* Error! */  
strcpy(str, "hello"); /* Right */
```

- כמו כן לא ניתן להציב מחרוזת אחת לשנייה, לדוגמה:

```
char str1[6] = "hello", str2[6];  
str2 = str1; /* Error! */  
strcpy(str2, str1); /* Right */
```

- לא ניתן לבדוק שוויון מחרוזות ע"י האופרטור ==, לדוגמה:

```
if(str == "hello") /* Logical Error! */  
if (strcmp(str, "hello")==0) /* Right */
```

■ אם מחרוזת מוגדרת ע"י מצביע, ניתן להציב למחרוזת קבועה, לדוגמה:

```
char *str;  
str = "hello"; /* Right */  
strcpy(str, "hello"); /* Running error! */
```

בלי ועם המשפט הקודם!

```
char *str;  
strcpy(str, "hello"); /* Running error! */
```

זיכרון למחרוזת לא הוקצה!

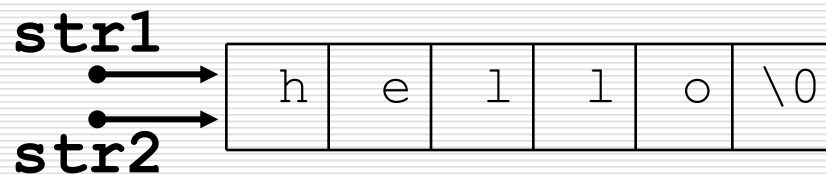
```
char *str;  
str = "hello";  
strcpy(str, "hello"); /* Running error! */
```

ניסיון לעדכן מחרוזת קבועה!

■ כמו כן ניתן להציב מצביע אחד לשני, לדוגמה:

```
char str1[6] = "hello", *str2;  
str2 = str1;
```

**אבל זו תהיה לא העתקת תוכן של מחרוזת אחת לשנייה אלא העתקת
מצביע אחד לשני!**



דוגמה לפונקציה strlen :

גרסה 1 :

```
int strlen1(char *s)
{
    int n;
    for(n=0;*(s+n)!='\0';n++);
    return n;
}
```

דוגמה לפונקציה strlen :

גרסה 2 :

```
int strlen2(char *s)
{
    int n;
    for(n=0; *s!='\0'; s++)
        n++;
    return n;
}
```

דוגמה לפונקציה strlen:

גרסה 3:

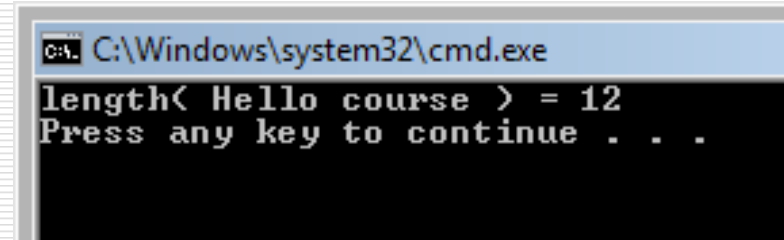
```
int strlen3(char *s)
{
    char *p=s;
    while (*p!='\0')
        p++;
    return p-s;
}
```

דוגמה לפונקציה strlen :

■ פונקציה strlen המקבלת מחרוזת ומחזירה אורכה :

```
int strlen4(char *s)
{
    int len=0;
    while(*(s++)!='\0') len++;
    return len;
}

void main()
{
    char *str="Hello course";
    printf("length( %s ) = %d\n", str, strlen4(str));
}
```



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\system32\cmd.exe'. The command prompt displays the output of the program: 'length< Hello course > = 12' followed by 'Press any key to continue . . .' on the next line.

דוגמה לפונקציה strcpy :

גרסה 1:

```
void strcpy1(char *s1, char *s2)
{
    int i;
    for ( i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ );
}
```

גרסה 2:

```
void strcpy2(char *s1, char *s2)
{
    for ( ; ( *s1= *s2 ) != '\0'; s1++,s2++ );
}
```

מניחים שגודל מערך s1 מספיק להעתקה לתוכו תוכן של מחרוזת s2.
אחרת – שגיאת ריצה!!

דוגמה לפונקציה strcmp :

פונקציה משווה שתי מחרוזות : היא מחזירה -1 אם מחרוזת ראשונה קטנה משנייה, מחזירה 0 אם שוויון, ומחזירה 1 אם מחרוזת ראשונה גדולה משנייה. ■

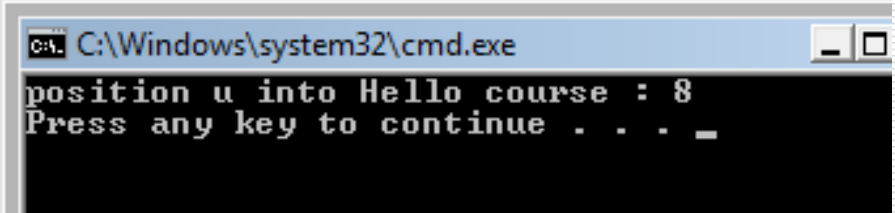
```
int strcmp(char *st1, char *st2) //name of standard function: strcmp
{
    int i;
    for(i=0; st1[i]!='\0' && st2[i]!='\0'; i++)
    {
        if( st1[i] < st2[i]) return -1;
        if( st1[i] > st2[i]) return 1;
    }
    if(st1[i]=='\0' && st2[i]!='\0') return -1;
    if(st1[i]!='\0' && st2[i]=='\0') return 1;
    return 0;
}
```

דוגמאות לפונקציות עיבוד מחרוזות

פונקציה `int strchr(char *s, char c)` מחזירה מקום הראשון של תו `c` בתוך מחרוזת `s`.

```
int strchr(char *s, char c)
{
    int pos=0;
    while( *s!='\0' )
        if(*s==c) return pos;
        else { pos++; s++;}
    return -1;
}
```

```
void main()
{
    char *str="Hello course";
    printf("position %c into %s : %d\n", 'u', str, strchr(str,'u'));
}
```



The screenshot shows a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The output of the program is displayed as follows:

```
position u into Hello course : 8
Press any key to continue . . . _
```

דוגמאות לפונקציות עיבוד מחרוזות

מימוש אותה פונקציה `int strchr(char *s, char c)` בעזרת מצביעים.

```
int strchrP(char *s, char c)
{
    char *pos=s;
    while( *pos!='\0' )
        if(*pos==c) return pos-s;
        else pos++;
    return -1;
}
```

פונקציות הספרייה string.h

- הפונקציות שממשנו לעיל הם פונקציות תקניות הקיימות בספרייה string.h בשפת C.

• strcmp השוואה בין 2 מחרוזות:

```
int strcmp(const char *s1, const char *s2);
```

• stricmp כמו strcmp אך ללא התחשבות בהבדלי אותיות גדולות/קטנות:

```
int stricmp(const char *s1, const char *s2);
```

• strcpy העתקת מחרוזת אחת למחרוזת שנייה:

```
char * strcpy(char *s1, const char *s2);
```

• strlen החזרת אורך מחרוזת:

```
size_t strlen(const char *s);
```

size_t הוא גודל טיפוס המוגדר ע"י typedef כשלם חיובי.

• strcat שרשור מחרוזת אחת למחרוזת שנייה:

```
char * strcat(char *dest, const char *src);
```